Python Assignment

¿Cuál es la diferencia entre una lista y una tupla en Python?

Las diferencias entre una lista y una tupla es que la lista va en corchetes [] y la tupla en paréntesis (), además la lista se puede modificar ya que es mutable, pero las tuplas no se pueden modificar ya que son inmutables.

Ejemplo:

Lista

```
frameworks = ["Django", "Flask", "FastAPI"]
```

Ej:

Tupla

```
python_info = ("Python", "Guido van Rossum", "1991")
```

La diferencia más importante es la mutabilidad.

Lo que significa que la lista ya que es mutable después de ser creada, se pueden añadir y eliminar elementos, así como cambiar y reordenarla.

Las listas se utilizan cuando necesitas que los datos cambien y necesitas añadir y eliminar elementos, por ello se usa para colecciones cambiantes

A diferencia de la Lista la Tupla es inmutable no se puede modificar después de su creación, no es posible añadir, eliminar o modificar los elementos. La única forma de cambiarla es crear una nueva tupla, por ello se usa para colecciones que representas datos fijos.

Las tuplas se utilizan cuando no necesitas que los datos cambien y quieres garantizar la inmutabilidad de los datos

Además, las tuplas dependen de una coma aparte de ir entre paréntesis llevan una coma al final

Tupla de un elemento

python_info =("Python",)

¿Cuál es el orden de las operaciones?

El orden de las operaciones es

1. Paréntesis

2. Exponente

3.Multiplicacion y División, tienen la misma prioridad de izquierda a derecha

5. Suma y la resta, tienen la misma prioridad de izquierda a derecha

Ejemplo:

5 + 3 * 4 – (6+5) ** 2

1. Sumamos el paréntesis (6+5) = 11

2. Lo elevamos a la 2 potencia 11**2 = 121

3. Ahora hacemos la multiplicación 3*4 = 12

4. Hacemos la suma 5 + 12 =17

5. Hacemos la resta 17 -121 = -104

Es el conjunto de reglas que determinan en que orden se tiene que hacer la operaciones dentro de una expresión cuando hay varios operadores diferentes.

Hay una regla mnemotécnica para poder ayudar a recordar el orden .

PEMDAS significa:

P - Parentheses
E - Exponents
M - Multiplication
D - Division
A - Addition
S - Subtraction

¿Qué es un diccionario Python?

Es una colección de datos que almacena pares de clave y valor, que va entre llaves {}

Ej:

```
"Python": {
        "year": 1991,
        "creator": "Guido van Rossum",
        "type": "interpretado",
        "paradigm": "multiparadigma",
        "popular_use": "data science, IA, automatización",
        "extension": ".py",
        "frameworks": ["Django", "Flask", "FastAPI"]
         }
```

Cada clave se utiliza como un identificador único para acceder al valor asociado. En un diccionario no se puede acceder a los datos por posición, sino por clave eso hace que la búsqueda sea muy rápida.

Estas claves deben de ser string, números o tuplas no pueden ser listas o otros diccionarios.

Las ventajas de usar un diccionario es que la búsqueda es muy rápida, organiza los datos semánticamente, almacenas estructuras complejas y son muy flexibles.

Los Diccionarios son dinámicos ya que se pueden añadir o eliminar elementos, también son indexados se pueden acceder a través de la clave (key) y anidado ya que puede contener otros diccionarios
Para acceder a un diccionario se hará mediante corchetes.
Ej :

```
Python[year]      =>  esto dará 1991
```

Si la clave no existe dará error y evitar que el programa se rompa se usara get
Ej:

```
print(Python.get('libraries', 'No found')
```

Entre los métodos de los diccionarios se encuentran key(), values(), e items()

El método key() devuelve todas la claves del diccionario, que son los identificadores que se usa para acceder a los valores devuelve un objeto llamado dict_keys, si el diccionario cambia también cambiara las vistas automáticamente, se usa cuando necesitas acceder solo a las claves.

El método values() devuelve todos los valores del diccionario en un objeto dict_values, se usa cuando solo necesitas acceder a las datos sin importar las claves.

El método ítem() devuelve las claves y los valores juntos en un objeto dict_items, cada elemento lo devuelve como una tupla se usa cuando necesitas obtener ambos elementos juntos .

¿Cuál es la diferencia entre el método ordenado y la función de ordenación?

El método ordenado, ordena, pero no devuelve nada.

Por ejemplo:

```
programming_languages = ["Python", "Java", "C++", "JavaScript", "Ruby",
"Go","Kotlin"]

programming_languages.sort() print(programming_languages)
```

Resultado lo ordena

```
['C++', 'Go', 'Java', 'JavaScript', 'Kotlin', 'Python', 'Ruby']
```

si lo quieres guardar en una variable de devuelve None

```
programming_ordered =programming_languages.sort() print(programming_ordered)
```

En cambio, la función de ordenación si que permite almacenar el resultado en un variable la función ordena y nos devuelve una nueva lista ordenada en este caso.

Ej.:

```
programming_languages = ["Python", "Java", "C++", "JavaScript", "Ruby", "Go","Kotlin"]
progrmaming_ordered = sorted(programming_languages) print(progrmaming_ordered)
```

devuelve

```
['C++', 'Go', 'Java', 'JavaScript', 'Kotlin', 'Python', 'Ruby']
```

Ahora tenemos dos listas una la original y otra la que hemos creado con los elementos ordenados .

La diferencia fundamental es como actúan sobre los datos, si modifica el objeto original o si crea un nuevo y en que estructuras se usa. Ambos ordenan, pero se comportan de diferente manera.

El método Sort() ordena la lista dentro de la misma estructura, esto se llama ordenacion “in place” (en el mismo lugar de la memoria). No crea una nueva lista , ni altera el contenido real. Se usa cuando necesitas conservar el orden original de los datos. Ya que es un método de lista y esta puede ser modificada.

La función Sorted() es no modifica el objeto original, toma los elementos iterables en este caso de una lista, los ordena y los devuelve en una nueva lista ordenada ya que sorted() se puede utiliza con cualquier colección de datos.

¿Qué es un operador de reasignación?

Es un operador el cual actualiza el valor de la variable utilizando su valor actual y además una operación aritmética y guarda el resultado en la misma variable. Por ejemplo, si queremos sumar 5 a una variable lo que hace es coger esa variable le suma 5 y además lo guarda el resultado en la misma variable.

Por ejemplo:

variable = 10

variable += 5

esto guardara en la variable 15 lo que es lo mismo

variable = variable + 5

Tipos de operadores que hay en Python son:

+=	Suma y asigna: x += 3 (equivale a x = x + 3)
-=	Resta y asigna: x -= 1 (equivale a x = x - 1)
*=	Multiplica y asigna: x *= 2 (equivale a x = x * 2)
/=	Divide y asigna x /= 2 (equivale a x = x / 2)
//=	Divide y asigna y devuelve el número entero y desecha decimales x //= 2 (equivale a x = x // 2)
**=	Eleva a la potencia y asigna x **= 2 (equivale a x = x ** 2)
%=	Calcula modulo y asigna x %=2(equivale a x = x % 2)

La reasignación se produce cuando una variable ya tiene un valor y recibe uno nuevo con los operadores de reasignación, que son la forma abreviada de realizar la actualización, lo que conlleva una ventaja al usarlos ya que hace que el código sea más corto.